

■ COMPLETE NOTES - TOPIC 13

CSS Media Queries

Responsive design ke liye screen size, device aur user preference ke according CSS change karna

Web Development Course - Shauray

@media

responsive design

breakpoints

min-width

max-width

mobile-first

desktop-first

orientation

media types

user preferences

Media Queries Kya Hote Hain?

Condition ke basis par CSS apply karna

SIMPLE DEFINITION

`@media` CSS rule browser ko condition check karne bolta hai. Agar condition true hoti hai, to uske andar likhi CSS apply hoti hai. Iska most common use responsive design mein hota hai, jahan mobile, tablet aur desktop ke liye layout adjust karte hain.

Aaj websites sirf ek screen size par nahi chalti. Same page phone, tablet, laptop, desktop, TV aur print page par open ho sakta hai. Media queries CSS ko smart banati hain: screen chhoti ho to cards one column, screen badi ho to multiple columns, print ho to background remove, user dark mode prefer kare to theme change.

basic media query

```
.container {  
  padding: 16px;  
}  
  
@media (min-width: 768px) {  
  .container {  
    padding: 32px;  
  }  
}
```

■ Media Query Ka Basic Formula

Part	Meaning
<code>@media</code>	Media query rule start karta hai
<code>condition</code>	Jaise min-width, max-width, orientation, print
<code>{ CSS }</code>	Condition true hone par ye CSS apply hoti hai

■ **Core Idea:** Media query CSS ko responsive banati hai, but responsive design ka base still flexible layout, relative units, images aur good content structure hota hai.

Viewport Meta Tag

Responsive CSS ke liye HTML head mein required setup

IMPORTANT SETUP

Responsive design ke liye HTML document ke `head` mein viewport meta tag hona chahiye. Ye browser ko batata hai ki page width device width ke equal treat kare aur initial zoom normal rakhe.

viewport meta tag

```
<meta name="viewport"
      content="width=device-width, initial-scale=1.0">
```

■ Without Viewport Meta Problem

Without Meta	With Meta
Mobile browser page ko desktop-like width mein render kar sakta hai	Page actual device width ke according render hota hai
Media queries unexpected behave kar sakti hain	Breakpoints predictable hote hain
User ko zoom/pan karna pad sakta hai	Content mobile-friendly scale par dikhta hai

■ **Must Remember:** Media queries likhne se pehle viewport meta tag ensure karo. Ye responsive pages ka foundation hai.

min-width vs max-width

Breakpoint condition kis direction mein apply hogi

MAIN DIFFERENCE

min-width ka matlab hai: screen at least itni width ho tab CSS apply karo. **max-width** ka matlab hai: screen itni width ya usse chhoti ho tab CSS apply karo.

MIN-WIDTH

Mobile-first approach mein common.
Base CSS small screen ke liye.
Larger screens par styles add karo.

MAX-WIDTH

Desktop-first approach mein common.
Base CSS large screen ke liye.
Smaller screens par styles override karo.

min-width and max-width

```
/* 768px and above */
@media (min-width: 768px) {
  .grid { grid-template-columns: repeat(2, 1fr); }
}

/* 767px and below */
@media (max-width: 767px) {
  .sidebar { display: none; }
}
```



■ **Practical Tip:** Ek project mein mostly ek approach follow karo. Mix karna possible hai, but beginners ke liye mobile-first **min-width** cleaner hota hai.

Mobile-First Approach

Pehle small screen CSS, phir larger screens ke liye enhance karna

Mobile-first ka matlab hai base CSS mobile/small screen ke liye likho. Phir `min-width` media queries se tablet aur desktop layout add karo. Ye approach modern responsive design mein common hai kyunki small screens par content priority clear hoti hai.

● ● ● mobile first layout

```
.cards {  
  display: grid;  
  grid-template-columns: 1fr;  
  gap: 16px;  
}  
  
@media (min-width: 768px) {  
  .cards {  
    grid-template-columns: repeat(2, 1fr);  
  }  
}  
  
@media (min-width: 1024px) {  
  .cards {  
    grid-template-columns: repeat(3, 1fr);  
  }  
}
```

RESPONSIVE CARDS CONCEPT

Mobile: one column base layout.

Tablet: two columns after breakpoint.

Desktop: three columns after wider breakpoint.

■ Why Mobile-First Better Hai?

Reason	Explanation
Cleaner CSS	Small screen base simple hota hai; large screen enhancements add hote hain
Performance thinking	Mobile users ke constraints pehle consider hote hain
Progressive enhancement	Basic experience sabko milta hai, bigger screens par richer layout

Desktop-First Approach

Pehle large screen CSS, phir smaller screens ke liye reduce karna

DEFINITION

Desktop-first approach mein base CSS desktop/large screens ke liye hoti hai. Phir `max-width` media queries se tablet aur mobile ke liye styles override karte hain.

desktop first layout

```
.layout {  
  display: grid;  
  grid-template-columns: 260px 1fr;  
  gap: 24px;  
}  
  
@media (max-width: 767px) {  
  .layout {  
    grid-template-columns: 1fr;  
  }  
  
  .sidebar {  
    display: none;  
  }  
}
```

■ Mobile-First vs Desktop-First

Approach	Base CSS	Query Type
Mobile-first	Small screens	min-width
Desktop-first	Large screens	max-width

■ **Real World:** Existing old websites often desktop-first hoti hain. New projects mein mobile-first usually better starting point hota hai.

Breakpoints

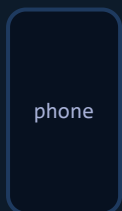
Screen width jahan layout change hota hai

DEFINITION

Breakpoint woh width hoti hai jahan design ko change karne ki zarurat padti hai. Example: 480px par mobile spacing, 768px par tablet columns, 1024px par desktop navigation.

■ Common Breakpoint Examples

Breakpoint	Common Meaning	Use
480px	Large phones	Small spacing/text adjustments
640px	Small tablets	Two-column small grids
768px	Tablet	Navigation/layout changes
1024px	Laptop	Desktop grid/sidebar
1280px	Large desktop	Max-width, wider spacing



BREAKPOINT THINKING

Breakpoint device ke naam par blindly choose mat karo. Design ko resize karke dekho: jahan content cramped, broken ya unreadable ho raha hai, wahan breakpoint useful hai.

■ **Best Practice:** Content-based breakpoints choose karo, device-based nahi. Devices bahut variety mein aate hain; design ka break point content decide kare.

Media Features

Width ke alawa aur conditions bhi check kar sakte hain

Media query sirf width ke liye nahi hoti. Browser device orientation, height, pointer type, hover support, color scheme preference, reduced motion preference aur print mode jaisi conditions bhi check kar sakta hai.

Feature	Checks	Example Use
orientation	portrait ya landscape	Game/UI layout adjust
height	Viewport height	Small laptop height adjustments
hover	Device hover support karta hai?	Desktop hover effects only
pointer	Touch vs precise pointer	Larger touch targets
prefers-color-scheme	User dark/light mode prefer karta hai?	Automatic theme
prefers-reduced-motion	User motion reduce prefer karta hai?	Animations reduce/remove

media feature examples

```
@media (orientation: landscape) {  
  .hero { min-height: 80vh; }  
}  
  
@media (hover: hover) {  
  .card:hover { transform: translateY(-4px); }  
}
```

Media Types: screen and print

Different output medium ke liye CSS change karna

DEFINITION

Media type batata hai CSS kis type ke output ke liye apply hogi. Common types `screen` aur `print` hain. Screen normal devices ke liye, print paper/PDF output ke liye use hota hai.

print styles

```
@media print {  
  body {  
    background: white;  
    color: black;  
  }  
  
  .navbar,  
  .footer {  
    display: none;  
  }  
}
```

■ When Print CSS Useful?

Document	Print CSS Need
Invoice / receipt	Buttons/sidebar hide, clean page layout
Notes / PDFs	Page breaks, print colors, readable spacing
Article	Navigation remove, links readable, paper-friendly type

■ **Same Notes Example:** Is HTML notes series mein bhi `@media print` use ho raha hai taaki PDF output clean page breaks aur exact colors ke saath aaye.

User Preference Queries

Dark mode aur reduced motion respect karna

Modern CSS user ke system preferences read kar sakta hai. Agar user dark mode prefer karta hai, to dark colors apply ho sakte hain. Agar user motion sensitivity ki wajah se reduced motion prefer karta hai, to heavy animations disable karni chahiye.

■ prefers-color-scheme

● ● ● dark mode preference

```
:root {
  --bg: #ffffff;
  --text: #111827;
}

@media (prefers-color-scheme: dark) {
  :root {
    --bg: #0f172a;
    --text: #e5e7eb;
  }
}
```

■ prefers-reduced-motion

● ● ● reduced motion

```
@media (prefers-reduced-motion: reduce) {
  * {
    animation-duration: 0.01ms;
    animation-iteration-count: 1;
    scroll-behavior: auto;
  }
}
```

■ **Accessibility Tip:** Reduced motion preference respect karna polish nahi, accessibility hai. Motion-sensitive users ke liye heavy animation uncomfortable ho sakti hai.

and, comma and not

Multiple conditions combine karna

CONDITION LOGIC

Media queries mein multiple conditions combine kar sakte hain. **and** ka matlab dono conditions true honi chahiye. Comma ka matlab OR jaisa behavior hai. **not** condition ko negate karta hai.

● ● ● and condition

```
@media (min-width: 768px) and (max-width: 1023px) {  
  .layout {  
    grid-template-columns: repeat(2, 1fr);  
  }  
}
```

● ● ● comma means or

```
@media (max-width: 480px), (orientation: portrait) {  
  .toolbar {  
    flex-direction: column;  
  }  
}
```

Operator	Meaning
and	All conditions true honi chahiye
,	Koi bhi condition true ho to CSS apply
not	Condition ko reverse karna

Real Project Patterns

Navbar, grid, typography aur images ko responsive banana

■ Responsive Navbar

● ● ● navbar pattern

```
.nav-links {  
  display: none;  
}  
  
.menu-button {  
  display: inline-flex;  
}  
  
@media (min-width: 768px) {  
  .nav-links {  
    display: flex;  
  }  
  
  .menu-button {  
    display: none;  
  }  
}
```

■ Responsive Typography

type scale

```
h1 {  
  font-size: 2rem;  
}  
  
@media (min-width: 900px) {  
  h1 {  
    font-size: 3.5rem;  
  }  
}
```

■ Responsive Image Grid

image grid

```
.gallery {  
  display: grid;  
  grid-template-columns: 1fr;  
  gap: 12px;  
}  
  
@media (min-width: 640px) {  
  .gallery { grid-template-columns: repeat(2, 1fr); }  
}  
  
@media (min-width: 1024px) {  
  .gallery { grid-template-columns: repeat(4, 1fr); }  
}
```

Common Mistakes and Fixes

Media query bugs ko quickly identify karna

MISTAKE

Viewport meta tag bhool jana.
Too many random breakpoints banana.
Mobile-first aur desktop-first mix karna without plan.
Fixed widths use karna responsive layout mein.
Images ko max-width: 100% na dena.
Hover effects touch devices par depend karna.

FIX

Head mein viewport meta tag add karo.
Content-based breakpoints choose karo.
Ek primary strategy follow karo.
Flexible units, grid/flex and max-width use karo.
img { max-width: 100%; height: auto; } use karo.
hover media feature se hover-only styles apply karo.

■ Debugging Checklist

Check	Question
Viewport	Kya viewport meta tag present hai?
Breakpoint	Kya condition true ho rahi hai at current width?
Cascade	Kya later CSS earlier media query ko override kar rahi hai?
Specificity	Kya selector specificity expected hai?
Units	Kya layout fixed px width se break ho raha hai?
Testing	Kya mobile, tablet, desktop widths test kiye?

■ **Big Rule:** Media queries layout ko fix karne ka last-minute patch nahi hain. Pehle flexible design banao, phir breakpoints se polish karo.

CSS Media Queries - Complete Summary

Revision ke liye all important responsive concepts ek jagah

Concept	Meaning	Example
@media	Conditional CSS rule	@media (min-width: 768px)
min-width	Width equal ya bigger ho to apply	Mobile-first
max-width	Width equal ya smaller ho to apply	Desktop-first
breakpoint	Layout change point	768px, 1024px
orientation	Portrait/landscape check	@media (orientation: landscape)
print	Print/PDF CSS	@media print
prefers-color-scheme	User dark/light preference	@media (prefers-color-scheme: dark)
prefers-reduced-motion	User motion preference	Reduce animations

■ Fast Decision Guide

Need	Choose
Mobile se desktop tak layout grow karna	Mobile-first + min-width
Existing desktop design ko mobile fix karna	Desktop-first + max-width
Tablet-only style	min-width and max-width together
Print/PDF layout	@media print
Dark mode follow karna	prefers-color-scheme
Animations accessible banana	prefers-reduced-motion

■ Best Practices - Yaad Rakhna

DO

Viewport meta tag add karo.
 Mobile-first approach practice karo.
 Content-based breakpoints choose karo.
 Flexible layouts use karo: flex/grid/relative units.
 Images responsive banao.
 Real device widths par test karo.

AVOID

Har device ke liye separate breakpoint banana.
 Random max-width/min-width mix karna.
 Fixed pixel layout par depend karna.
 Accessibility preferences ignore karna.
 Hover-only UI touch users ke liye banana.
 Media queries ko messy override dumping ground banana.

> NEXT TOPICS TO COVER

Responsive Design

Flexbox

CSS Grid

Container Queries

Transitions

Animations

Forms Styling

Pseudo Classes

Made with ❤️ for Web Development Course learners

Shauray ke saath CSS seekhte raho 🚀 | Topic 13 - CSS Media Queries